

Back Off and Give Up

Moshe Zadka – <https://cobordism.com>

Acknowledgement of Country

Hayward (in San Francisco Bay Area)

Ancestral homeland of the Ohlone people

I live in Hayward, in the San Francisco Bay Area. I wish to acknowledge it as the ancestral homeland of the Ohlone people.

The Secret to Resilient Systems

Quit

What if I told you that the secret to building resilient systems is learning when to quit? This might sound counterintuitive - we're taught that persistence is a virtue. But in distributed systems, sometimes giving up is exactly the right strategy. Sometimes, trying harder makes things worse.

Naive Retries

The problem

Part 1: The Problem with Naive Retries

Naive Retry: The Classic Mistake

```
import time
import requests

def get_data(url):
    while True:
        try:
            response = requests.get(url, timeout=5)
            return response.json()
        except requests.RequestException:
            time.sleep(1)
```

Here's code we've all written. Simple retry loop - if it fails, wait a second and try again. Forever. This has infinite retries, fixed delay, no backoff, no jitter, no maximum. When the service recovers, every client hits it simultaneously.

Thundering Herd

Recover

Crash

The thundering herd problem. Service goes down. A thousand clients retry every second. Service recovers after 60 seconds. A thousand simultaneous connections hit it. The service crashes again immediately. This is what happened in our recipe site story.

Retry Storms

Frontend → API Gateway → Payment Service

3

×3

= 9

$9 \times 100 = 900$

Retry storms. Payment processing service with three layers. Frontend, API Gateway, Payment Service. Each layer has 3 retries. One failed request becomes 27 attempts. At 100 requests per second, that's 2,700 attempts per second. The struggling service gets more load.

Back-off

Do better!

Part 2: Exponential Back-off with Jitter

Exponential

Wait

Proportional to outage

Exponential backoff reduces load quickly as failures continue. Gives the service time to recover. Self-regulating - worse problems get longer waits. Wait time grows with the severity of the problem.

Basic Implementation

```
wait_time = 2 ** attempt
time.sleep(wait_time)
```

Basic exponential backoff implementation. Wait 2 to the power of attempt seconds. Wait times: 1, 2, 4, 8, 16 seconds. But everyone's still synchronized - they all retry at the same times.

Jitter

Without jitter: Everyone at T=4s

Jitter is critical. Without jitter, all clients retry at exactly 1, 2, 4 seconds. With jitter, we spread them across time windows. Full jitter picks a random time between 0 and the calculated backoff. Instead of everyone at 4 seconds: Client A at 3.2, Client B at 1.8, Client C at 3.9. The load spreads naturally. This randomization prevents the thundering herd.

Jitter

```
time.sleep(
    random.uniform(0.9, 1.1)
    *
    2**attempt
)
```

Jitter is critical. Without jitter, all clients retry at exactly 1, 2, 4 seconds. With jitter, we spread them across time windows. Full jitter picks a random time between 0 and the calculated backoff. Instead of everyone at 4 seconds: Client A at 3.2, Client B at 1.8, Client C at 3.9. The load spreads naturally. This randomization prevents the thundering herd.

A Tale of Two Worlds: Premise

Recipe recommendation site with ML backend

A Tale of Two Worlds: World 1

Max backoff = 1 second

A Tale of Two Worlds: World 2

Max backoff = 10 minutes

Let me tell you a story about a recipe recommendation site with an ML backend. During a model rollout, nodes started becoming flaky. In one world - let's call it World 1 - the max backoff was set to 1 second. The front-ends hammered the ML nodes relentlessly, causing a 3-hour complete outage that made the news. But in World 2, where the max backoff was 10 minutes, the system recovered gracefully with just minor degradation that nobody even noticed. One configuration parameter. Two completely different outcomes.

Tenacity

```
@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(
        max=600 # 5 minutes!
    )
)
```

Production code using tenacity. Stop after 5 attempts. Exponential backoff with max of 300 seconds - that's 5 minutes. This is the parameter that saved World 2 in our story.

Giving Up

Tactical retreat

Part 3: Strategic Giving Up - this is the counterintuitive part.

Why Give Up?

Avoid catastrophizing

Persistent retrying prevents recovery. Resources tied up in doomed requests. Failures cascade. Strategic abandonment means quick failure, quick recovery. Resources stay available for healthy operations. Partial service beats no service. Sometimes the best thing for a struggling system is to stop using it.

Circuit Breaker

Three states:

Closed

Open

Half-Open

Circuit breaker pattern - like an electrical circuit breaker. Three states. Closed: normal operation, requests flow through. Open: too many failures, reject immediately without trying. Half-open: test if service recovered with limited traffic. Fails fast when service is down. Automatic recovery detection. Prevents cascade failures.

PyBreaker

```
@CircuitBreaker(
    fail_max=5,
    reset_timeout=60
)
def get_user_data(user_id):
    return database.query(...)
```

PyBreaker implementation. Opens after 5 failures. Tries recovery after 60 seconds. Exclude application errors like KeyError - only trip on infrastructure failures. When open, use a fallback. Degraded service beats no service.

Load Shedding

Priority levels

Load shedding - choose what to drop. Critical: payments and auth must continue. Important: user updates and searches. Nice-to-have: ML recommendations. Background: reports. At 70% capacity, delay background jobs. 80%, disable recommendations. 90%, simplify search - basic keywords only. 95%, read-only mode. The recipe site could have kept basic search while shedding ML recommendations.

Optional Features

```
@optional_feature(fallback=[])
def get_recommendations(user_id):
    return ml_model.predict(user_id)
```

Decorator for optional features. Above 80% load, return fallback immediately. Aggressive timeout - half second. The recommendations function returns empty list when stressed. Users still get the site.

Putting It Together

Configuration time

Part 4: Putting It All Together

Real Configuration

```
retry:
  max_delay: 300 # 5 minutes!
  jitter: true

circuit_breaker:
  failure_threshold: 5
  recovery_timeout: 60

load_shedding:
  0.7: ["reports"]
  0.8: ["recommendations"]
  0.9: ["non_critical"]
```

Real configuration. Max delay: 300 seconds, 5 minutes, human timescale. Jitter enabled. Circuit breaker: 5 failures to open, 60 seconds recovery. Progressive load shedding: 70% drops reports, 80% drops recommendations, 90% drops non-critical. Different timeouts: 5 seconds default, 1 second critical path, 0.5 seconds optional features.

Monitor Everything

Metrics

Alerts

Monitor everything. Track retry success rates - low success means something's wrong. Circuit breaker states - which services struggle. Load shed counts - what functionality are we sacrificing. Alert when breakers stay open too long. Alert when retry success drops below 50%. Alert when load shedding activates. Dashboards need service health maps, retry ratios, load shedding timelines.

Key Takeaways

- Exponential backoff with jitter
- Max backoff: 1-10 minutes
- Give up
- Prioritize

Key points. Retries can make things worse - thundering herds, retry storms. Exponential backoff with jitter spreads load. Set max to human timescales: 1 to 10 minutes. Give up strategically - circuit breakers, load shedding. Partial service beats no service. Failing fast means recovering faster.

Resources

Libraries:

- `tenacity`
- `pybreaker`

Python libraries. Tenacity for comprehensive retries. PyBreaker for circuit breakers.

Questions?

Moshe Zadka

<https://cobordism.com>

Questions? Thank you!